

ELO329 - Diseño y Programación Orientados a Objetos

Funciones amigas y miembros estáticos en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

En las clases anteriores construimos clases, protegimos su estado con `private` y usamos `const` para dejar claras las promesas de no modificación.

Ahora veremos dos herramientas que ajustan esa interfaz:

- `friend` , para dar acceso excepcional a funciones o clases externas.
- `static` , para representar datos y operaciones asociadas a la clase completa.

La idea común es distinguir qué pertenece a un objeto individual y qué pertenece al diseño completo de la clase.

Contenidos

1. Encapsulamiento y acceso excepcional
2. Funciones `friend`
3. Sobrecarga de operadores con funciones globales
4. Clases amigas
5. Reglas de la amistad
6. Atributos estáticos
7. Métodos estáticos
8. Contadores de instancias

Punto de partida: encapsulamiento

Una clase normalmente mantiene sus atributos en `private`.

```
class Course {  
public:  
    bool isValid() const;  
  
private:  
    int nCredits;  
};
```

El usuario de la clase no accede directamente a `nCredits`.

Esto protege las reglas internas del objeto.

El problema práctico

A veces una operación no calza naturalmente como método de la clase.

Ejemplos:

- Una función global que valida o compara objetos.
- Un operador donde el operando izquierdo no es nuestra clase.
- Una clase auxiliar que necesita inspeccionar detalles internos.

La pregunta de diseño es:

¿Mantengo todo privado o autorizo una excepción acotada?

Funciones friend

Una función `friend` es una función externa autorizada por la clase para acceder a sus miembros `private` y `protected`.

```
class Course {  
public:  
    friend bool validateCourseData(const Course &c);  
  
private:  
    int nCredits;  
};
```

La declaración está dentro de la clase, pero la función no se convierte en método.

Lectura de la declaración

```
friend bool validateCourseData(const Course &c);
```

Lectura:

- `validateCourseData` sigue siendo una función global.
- `Course` decide confiar en esa función.
- La función puede leer o modificar miembros privados, según reciba `const` o no.

`friend` no rompe el encapsulamiento desde fuera; la propia clase concede el permiso explícitamente.

Implementación de una función friend

La palabra `friend` aparece solo en la declaración dentro de la clase.

```
bool validateCourseData(const Course &c) {  
    return c.nCredits >= 1 && c.nCredits <= 5;  
}
```

Aquí se accede a `nCredits`, aunque sea privado.

Eso es posible porque `course` declaró esta función como amiga.

Cuidado con el alcance

Una función amiga no pertenece a la clase.

```
validateCourseData(course); // ok  
course.validateCourseData(); // error
```

Esta diferencia es importante: `friend` otorga acceso, no cambia la forma de llamar la función.

El caso más frecuente donde esto aparece es la sobrecarga de operadores.

Operadores como métodos

Si el operando izquierdo es un objeto de nuestra clase, el operador puede escribirse como método.

```
class CVector {  
public:  
    CVector operator+(const CVector &other) const;  
  
private:  
    double x, y;  
};
```

Operadores como métodos

Entonces:

```
c = a + b;
```

se interpreta como:

```
c = a.operator+(b);
```

Implementación de operator+

```
CVector CVector::operator+(const CVector &other) const {  
    CVector temp;  
    temp.x = x + other.x;  
    temp.y = y + other.y;  
    return temp;  
}
```

Hay dos promesas:

- `other` se recibe por referencia constante.
- La suma no modifica el objeto izquierdo.

El resultado es un nuevo `CVector`.

Cuando el método no basta

Queremos permitir:

```
CVector v2 = 3.0 * v1;
```

Si `operator*` fuera método de `CVector`, la forma natural sería:

```
v1.operator*(3.0)
```

Eso corresponde a:

```
v1 * 3.0
```

Pero en `3.0 * v1`, el operando izquierdo es un `double`, no un `CVector`.

Operador global como friend

La solución es una función global.

```
class CVector {  
public:  
    CVector(double, double);  
    friend CVector operator*(double factor, CVector v);  
  
private:  
    double x, y;  
};
```

Ahora la función global puede construir el resultado usando los datos privados de `v`.

Implementación de operator*

```
CVector operator*(double factor, CVector v) {  
    return CVector(factor * v.x, factor * v.y);  
}
```

La función no es método, pero puede acceder a `x` e `y` por ser `friend`.

Este patrón se usa cuando la sintaxis deseada no puede expresarse naturalmente con un método de la clase.

operator<< y flujos

Otro caso típico es imprimir objetos.

```
std::cout << v;
```

El objeto izquierdo es `std::cout`, de tipo `std::ostream`.

No podemos modificar la clase `std::ostream` para agregarle métodos de nuestras clases.

Por eso se usa una función global.

Sobrecarga de operator<<

```
class CVector {  
public:  
    friend std::ostream &operator<<(std::ostream &os,  
                                    const CVector &v);  
  
private:  
    double x, y;  
};
```

La función retorna `std::ostream &` para permitir encadenamiento.

```
std::cout << "v = " << v << std::endl;
```

Implementación de operator<<

```
std::ostream &operator<<(std::ostream &os,  
                        const CVector &v) {  
    os << "(" << v.x << "," << v.y << " )";  
    return os;  
}
```

`os` se retorna para que la siguiente operación `<<` siga escribiendo sobre el mismo flujo.

friend también puede concederse a una clase completa, no solo a una función.

Clases amigas

Una clase puede declarar que otra clase completa es amiga.

```
class A {  
public:  
    friend class B;  
  
private:  
    int x, y;  
};
```

Con esa declaración, los métodos de **B** pueden acceder a los miembros privados de **A**.

Uso de una clase amiga

```
class B {  
public:  
    int sum(const A &a) const {  
        return a.x + a.y;  
    }  
};
```

La autorización la entrega **A**, no **B**.

Esto mantiene una relación explícita: **A** está diciendo qué otra clase puede conocer sus detalles internos.

Reglas de la amistad

La amistad en C++ es acotada.

Regla	Consecuencia
No es mutua	Si <code>A</code> declara amiga a <code>B</code> , <code>A</code> no accede automáticamente a privados de <code>B</code>
No se hereda	Una clase derivada de <code>B</code> no hereda el permiso
No es transitiva	Si <code>B</code> es amiga de <code>C</code> , <code>A</code> no pasa a confiar en <code>C</code>

Esto evita que un permiso local se propague sin control.

Cuándo usar friend

Usar `friend` cuando la operación:

- Realmente necesita acceder a detalles internos.
- No pertenece naturalmente como método.
- Mejora la interfaz pública en lugar de agrandarla artificialmente.

Evitarlo cuando bastan métodos públicos bien diseñados.

`friend` es una decisión de diseño, no un atajo para evitar encapsular.

Cambio de foco: static

Hasta ahora, cada objeto tenía su propio estado.

```
Student a;  
Student b;
```

Cada estudiante puede tener datos propios.

Pero a veces necesitamos un dato compartido por todos los objetos de la clase.

```
¿Cuántos Student existen ahora?
```

Atributos estáticos

Un atributo `static` pertenece a la clase, no a cada objeto.

```
class Student {  
private:  
    static int m_snCount;  
};
```

Todas las instancias comparten el mismo `m_snCount`.

No hay una copia por objeto.

Declaración versus definición

La línea dentro de la clase declara que el atributo existe.

```
class Student {  
private:  
    static int m_snCount;  
};
```

Pero también se necesita una definición en un `.cpp`.

```
int Student::m_snCount = 0;
```

Esa definición reserva almacenamiento e inicializa el valor.

Por qué no va en el constructor

El atributo estático existe antes de crear cualquier objeto.

Por eso no se inicializa así:

```
Student::Student() {  
    m_snCount = 0; // mala idea para un contador global  
}
```

Si cada constructor reiniciara el contador, se perdería la cuenta acumulada.

La inicialización se hace una vez, fuera de los constructores.

Contador de instancias

El constructor y destructor pueden mantener el contador actualizado.

```
Student::Student() {  
    m_snCount++;  
}  
  
Student::~~Student() {  
    m_snCount--;  
}
```

Cada objeto creado incrementa el valor compartido.

Cada objeto destruido lo decrementa.

Métodos estáticos

Un método `static` también pertenece a la clase.

```
class Student {  
public:  
    static int getInstanceCount();  
  
private:  
    static int m_snCount;  
};
```

Puede llamarse sin tener un objeto.

```
std::cout << Student::getInstanceCount();
```

Restricción importante

Un método estático no tiene `this`.

Por eso solo puede acceder directamente a miembros estáticos.

```
int Student::getInstanceCount() {  
    return m_snCount;  
}
```

No puede leer atributos de un estudiante particular, porque no hay estudiante particular asociado a la llamada.

Llamadas a métodos estáticos

La forma recomendada es usar el nombre de la clase.

```
std::cout << Student::getInstanceCount();
```

C++ también permite llamar desde una instancia.

```
Student s;  
std::cout << s.getInstanceCount();
```

Pero la primera forma comunica mejor que la operación pertenece a la clase completa.

Secuencia de uso

```
std::cout << Student::getInstanceCount(); // 0

Student s1;
Student s2;

std::cout << Student::getInstanceCount(); // 2
```

La cuenta no está almacenada en `s1` ni en `s2`.

Está almacenada una sola vez como miembro estático de `Student`.

Relación con Java

La idea conceptual es similar a Java:

```
class Student {  
    private static int count;  
}
```

La diferencia práctica en C++ es la definición separada:

```
int Student::m_snCount = 0;
```

Esta diferencia viene del modelo de compilación de C++ y de la separación entre `.h` y `.cpp`.

Conexión entre friend y static

Concepto	Pregunta que responde
<code>friend</code>	¿Quién puede ver detalles internos aunque no sea método?
<code>static</code>	¿Qué pertenece a la clase completa y no a cada objeto?

Ambos conceptos amplían la interfaz de una clase.

Por eso deben usarse para expresar una intención clara, no como solución automática.

Ideas clave

- `friend` concede acceso excepcional a miembros privados y protegidos.
- Una función amiga sigue siendo función global.
- `operator<<` y `factor * objeto` suelen implementarse como funciones globales.
- La amistad no es mutua, heredada ni transitiva.
- Un atributo `static` se comparte entre todas las instancias.
- Un dato estático debe definirse fuera de la clase.
- Un método estático no tiene `this`.
- Use `Clase::metodoEstatico()` para comunicar que la operación pertenece a la clase.